# 534. Game Play Analysis III

Table: `Activity`

```
+--------------+---------+
| Column Name  | Type    |
+--------------+---------+
| player_id    | int     |
| device_id    | int     |
| event_date   | date    |
| games_played | int     |
+--------------+---------+
(player_id, event_date) is the primary key of this table.
This table shows the activity of players of some game.
Each row is a record of a player who logged in and played a number of games (possibly 0) before logging out on some day using some device.
```

Write an SQL query that reports for each player and date, how many games played **so far** by the player. That is, the total number of games played by the player until that date. Check the example for clarity.

The query result format is in the following example:

```
Activity table:
+-----------+-----------+------------+--------------+
| player_id | device_id | event_date | games_played |
+-----------+-----------+------------+--------------+
| 1         | 2         | 2016-03-01 | 5            |
| 1         | 2         | 2016-05-02 | 6            |
| 1         | 3         | 2017-06-25 | 1            |
| 3         | 1         | 2016-03-02 | 0            |
| 3         | 4         | 2018-07-03 | 5            |
+-----------+-----------+------------+--------------+

Result table:
+-----------+------------+---------------------+
| player_id | event_date | games_played_so_far |
+-----------+------------+---------------------+
| 1         | 2016-03-01 | 5                   |
| 1         | 2016-05-02 | 11                  |
| 1         | 2017-06-25 | 12                  |
| 3         | 2016-03-02 | 0                   |
| 3         | 2018-07-03 | 5                   |
+-----------+------------+---------------------+
For the player with id 1, 5 + 6 = 11 games played by 2016-05-02, and 5 + 6 + 1 = 12 games played by 2017-06-25.
For the player with id 3, 0 + 5 = 5 games played by 2018-07-03.
Note that for each player we only care about the days when the player logged in.
```

## Difficulty:

Medium

## Lock:

Prime

## Company:

GSN Games

| Concept | Purpose |
| --- | --- |
| `SUM()` | Adds values |
| `OVER()` | Performs a window calculation |
| `PARTITION BY` | Separates calculations by player |
| `ORDER BY` | Determines chronological order |
| `ROWS BETWEEN` | Defines which rows are included |
| `UNBOUNDED PRECEDING` | Starts from the first row |
| `CURRENT ROW` | Ends at the current row |

DS | 6-11-26 | Day-9

Leetcode - 534, Game Play Analysis III.

We have an Activity table:

| player_id | device_id | event_date | games_played |
|---|---|---|---|
| 1 | 2 | 2016-03-01 | 5 |
| 1 | 2 | 2016-05-02 | 6 |
| 1 | 3 | 2017-06-25 | 1 |
| 3 | 1 | 2016-03-02 | 0 |
| 3 | 4 | 2018-07-03 | 5 |

We need to calculate the total number of games played by each player up to & including each date.

It means:-

→ on the 1st date, count games played on that date.

→ " " 2nd " , add the " from the 1st & 2nd date.

→ " " 3rd " , " games from all 3 dates.

So we need a running total (cumulative sum).

Ex

| player_id | event_date | games_played_so_far |
|---|---|---|
| 1 | 2016-03-01 | 5 |
| 1 | 2016-05-02 | 11 |
| 1 | 2017-06-25 | 12 |
| 3 | 2016-03-02 | 0 |
| 3 | 2018-07-03 | 5 |

Ans's day wise this,

player 1, running total for player 1,

1. 2016-03-01
   games played = 5, total 5.
2. 2016-05-02
   games played = 6
   total = 11
3. 2017-06-25
   game played = 1
   total = 5+6+1 = 12.

player 3, player 3 has a separate calc.
→ 1st date: 0 game → total = 0
→ 2nd date: 5 game → total = 0+5 = 5

player 3's total must not include player 1's games.

WINDOW FUNCTION

```
SELECT
    player_id,
    event_date,
    SUM(games_played) OVER (
        PARTITION BY player_id
        ORDER BY event_date
        ROWS BETWEEN UNBOUNDED
        PRECEDING AND CURRENT ROW
    ) AS games_played_so_far
FROM Activity;
```

we are telling SQL which columns we want in our final o/p player's ID, the event date /

this calculates the sum of game played.

we are giving the called column the name.

→ from activity table take info.

OVER() → tells SQL that we want to perform a window calculation. It's like calculate something while looking at a particular set of rows, without collapsing these rows into one.

→ this divides the data into separate groups based on player ID. Without this, SQL could add games from diff. players together.

→ but we want the running total in chronological order.

→ ROWS = looks with rows in the window

UNBOUNDED PRECEDING = start from the very 1st row of the current player's partition.

CURRENT ROW :- stop at the current row being calculated.

→ start from the player's 1st activity & keep adding games until the current date.

can we solve it without a window function?

```
SELECT
    a.player_id,
    a.event_date,
    (
     SELECT SUM(b.games_played)
     FROM Activity b
     WHERE b.player_id
        AND b.event_date <= a.event_date
    ) AS games_played_so_far
FROM Activity a;
```

→ a represent the current row.
→ b " rows we are checking for the same player
→ b.player_id = a.player_id ensures we only consider the same player.
→ b.event_date <= a.event_date ensure, we only add games played on or before the current day.
→ SUM() adds these games.